Milestone 1 · Oct 7, 2026, 12:00 (Beijing time)
第一阶段 · 2026年10月7日中午12:00（北京时间）
Submissions received by then will inform selection of collaboration proposals before the college project application deadline on Oct 10. / 届时收到的提交将用于遴选合作提议，以便在10月10日学院立项截止前推进。

Milestone 2 · Oct 12, 2026, 12:00 (Beijing time)
第二阶段 · 2026年10月12日中午12:00（北京时间）
Final homework submission deadline. / 作业最终提交截止时间。

▶ **Browse all resources**

Homework Phase Two: From Experimental Search to Mathematical Optimization

> **Phase Two opens October 2, 2026 at 11:00 Beijing time.** Existing Phase One submission fields and the October 12, 12:00 Beijing deadline are unchanged. The performance score is one part of the course assessment.

Read the technical statement · Get the starter package

Homework Phase Two: From Experimental Search to Mathematical Optimization
- Purpose
- 1. Updated Memory-Hierarchy Model
- 2. Learn from Phase One
- 3. Define the Optimization Problem
- 4. Use Knowledge to Make the Search Efficient
- 5. Build the Multi-Agent Framework
- 6. Verify and Improve the Formulation
- 7. Deliverables
- 8. Evaluation

Purpose

In Phase One, you used AI coding agents to explore accelerator designs and Transformer implementations. You tested different configurations and searched for better performance. These experiments produced both working solutions and knowledge about the design problem.

Phase Two asks you to turn that knowledge into a systematic optimization method.

Build a multi-agent framework that learns from your Phase One solutions, identifies the important design choices, and formulates them as a mixed-integer programming problem. Use a solver to search the resulting space, then use simulation to verify the generated solutions and improve the formulation.

The main learning objective is **to formulate and solve the hardware–software design problem efficiently**. Your final performance score is one outcome; the quality of your formulation and search process is also central.

---

1. Updated Memory-Hierarchy Model

Phase Two uses a revised model of the accelerator's memory hierarchy, covering:

- High-bandwidth memory (HBM).
- On-chip cache.
- On-chip shared memory.
- Registers.

The revision focuses on more realistic access latencies and transfer bandwidths. **Latency** describes how long an access takes. **Bandwidth** describes how much data can move per unit of time.

These characteristics affect the value of data reuse, computation tiling, data placement, and scheduling. For example, retaining a reusable input in shared memory can avoid repeated HBM transfers, but it also consumes local storage and introduces shared-memory accesses. A useful optimization model must represent the relevant costs and tradeoffs.

Use the published Phase Two parameters and simulator. Document any simplifications you introduce when translating the hardware model into a mathematical formulation.

---

2. Learn from Phase One

Start with your previous solutions and experimental records. Determine what they reveal about the problem:

- Which choices produced substantial performance changes?
- Which resources limited execution speed?
- Which choices worked well together?
- Why did unsuccessful configurations fail?
- Which observations may change under the revised memory model?

Convert these observations into explicit knowledge that your framework can use. Record the supporting evidence and the conditions under which each conclusion applies.

For example, “larger tiles are faster” is too broad. A more useful observation is: “Increasing this tile dimension reduced repeated input transfers, until the resulting storage requirement limited concurrent execution.”

Preserve your strongest previous solutions. Re-evaluate legal reference configurations under the revised model to establish a comparable baseline.

---

3. Define the Optimization Problem

Your agents must identify the **optimization knobs**: the permitted hardware and software choices that can affect performance.

Depending on the published assignment rules, these may include tile dimensions, data placement, buffer counts, work partitioning, and operation scheduling.

Translate those choices into a mathematical problem with four clearly explained components:

Component	What you must explain
Decision variables	What the solver can choose and the allowed values.
Constraints	What makes a configuration legal and preserves correct computation.
Performance model	How the choices determine predicted execution cost.
Objective	What quantity the solver seeks to improve.

A **mixed-integer programming (MIP) problem** uses numerical variables, some of which must take integer values. These integer variables can represent discrete design choices.

The formulation must expose meaningful configuration decisions to the solver. Selecting the best result from a list of already completed experiments is insufficient.

Illustrative example: choosing a computation tile

Suppose Phase One shows that repeated HBM transfers limit a matrix multiplication stage. Your framework might identify tile dimensions and buffer count as useful choices.

It would then:

1. Define the permitted tile dimensions and buffering options.
2. Exclude combinations that exceed local storage.
3. Estimate how each combination changes data reuse, transfer cost, and execution time.
4. Let the solver choose a configuration.
5. Generate that implementation and measure it in the simulator.

If the simulator reveals a cost missing from the prediction, the framework should revise the model before the next search.

This example illustrates the required reasoning process; it does not prescribe your formulation or require you to limit the framework to matrix multiplication.

---

4. Use Knowledge to Make the Search Efficient

A large search space is not automatically a useful one. Use Phase One knowledge to decide which choices matter, narrow their ranges, and remove combinations with a clear justification.

Distinguish three kinds of reasoning:

- **Infeasibility:** a combination violates a required constraint.
- **Dominance:** under stated assumptions, another configuration is demonstrably no worse, so the dominated choice can be excluded.
- **Heuristic pruning:** evidence suggests a region is unpromising, but excluding it may remove a good solution.

For heuristic pruning, explain the risk and how your framework can revisit the excluded region. Keep strong known solutions in the search space whenever they remain legal and supported.

The agents should also seek a formulation the solver can handle efficiently. They may tighten variable bounds, exploit problem structure, or divide the optimization into smaller subproblems. Explain how these decisions affect search cost and solution quality.

Explicit enumeration of every combination is not required. Report what the solver established: a feasible solution, an optimal solution within the encoded model, or a solution with an unresolved optimality gap.

---

5. Build the Multi-Agent Framework

The framework should connect the following responsibilities:

Responsibility	Required contribution
Knowledge extraction	Analyze Phase One evidence and identify useful mechanisms, bottlenecks, and uncertainties.
Formulation	Define variables, constraints, costs, and justified pruning rules.
Solver and execution	Invoke the MIP solver, generate implementations, and run simulations.
Verification and revision	Check correctness, compare predictions with observations, and revise the model or search space.

You may choose how to assign these responsibilities across agents. Explain what information they exchange, how they resolve conflicting conclusions, and how they preserve knowledge across iterations.

The framework must demonstrate that agent reasoning changes the formulation or search strategy. Running several independent brute-force searches in parallel does not by itself meet this requirement.

---

6. Verify and Improve the Formulation

A solver optimizes the mathematical problem it receives. Its result does not establish that the model accurately predicts the accelerator.

For each evaluated proposal:

1. Check that the generated implementation satisfies the assignment's correctness and eligibility requirements.
2. Compare predicted performance with simulated performance.
3. Investigate consequential discrepancies.
4. Update the formulation, pruning rules, or search priorities using the new evidence.

Keep model predictions separate from simulator measurements. A proof of optimality within a simplified model is not a proof that the implementation is the fastest possible design.

---

7. Deliverables

Submit:

As in Phase One, **local self-test results are required** for the exact final programs: the generated aggregate score and five case reports, with correctness/resource checks, seed and simulator/program identities. Include the reproduction command and host environment; disclose incomplete runs. See the self-test files and separate material-size allowances. Final scoring uses organizer re-evaluation.

- The executable multi-agent framework.

- The MIP formulation, including variable ranges, constraints, objective, and modeling assumptions.
- The knowledge extracted from Phase One and the resulting pruning rules.
- The generated hardware configuration and Transformer programs.
- Experimental records and a report explaining the results.

Include at least one complete example tracing:

Phase One evidence → optimization choices → pruning and formulation → solver proposal → simulation and verification → model revision.

---

8. Evaluation

Compare your framework with a documented Phase One search approach using the same revised simulator and comparable resource budgets.

Report:

- Final simulated performance and correctness.
- Number of simulator evaluations.
- Elapsed search time.
- AI token usage.
- Solver runtime and termination status.
- Agreement between predicted and simulated performance.

Explain whether the framework found better solutions, reached comparable solutions with less search effort, or revealed limitations in its formulation.

Assessment will consider performance, formulation quality, justified use of prior knowledge, effective use of the solver, cooperation between agents, search efficiency, and verification. The report should make clear **what the framework learned and how that knowledge improved its decisions**.

Continue to the changes, workloads and machine specification.

Agent-Guided Transformer Co-design